Thread in C

Primitive

```
int pthread_create (
        phtread_t* thread,
        const pthread_attr_t* attr,
        void* (*start_routine)(void*),
        void* arg);
```

- **thread**: puntatore a variabile di tipo **pthread_t,** su cui verrà memorizzato l'ID del thread creato;
- **attr**: attributi di creazione (sempre NULL in questo corso);
- **start_routine**: funzione da eseguire (prende sempre come argomento un void* e restituisce un void*);
- **arg**: puntatore da passare come argomento alla funzione start_routine;
- Ritorna 0 in caso di successo, altrimenti la causa dell'errore.

```
void pthread_exit (void* value_ptr);
```

- Termina il **thread corrente**, rendendo disponibile il valore puntato da **value_ptr** ad una eventuale operazione di join;
- All'interno di **start_routine**, può essere sostituita da **return**.

```
int pthread_join(phtread_t thread, void** value_ptr);
```

- Attende esplicitamente la terminazione del thread con ID **thread**;
- Se **value_ptr != NULL**, vi memorizza il valore eventualmente restituito dal thread (un **void***, tramite **pthread_exit**);
- Ritorna il valore 0 in caso di successo, altrimenti la causa dell'errore.

```
int pthread_detach(phtread_t thread);
```

- Notifica al sistema che non ci saranno operazioni di join su thread;
- Ritorna il valore 0 in caso di successo, altrimenti la causa dell'errore.

Esempio 1.

```
#include <stdio.h>
#include <errno.h>
#include <pthread.h>

void* thread_stuff(void *arg) {
    // codice thread che non usa argomenti
    return NULL;
}

int ret;
pthread_t thread;
ret = pthread_create(&thread,NULL,thread_stuff,NULL);
if (ret != 0) {
    fprintf(stderr, "ERROR with pthread_create!\n");
    exit(EXIT_FAILURE);
}
// codice main indipendente dal thread
ret = pthread_join(thread, NULL);
if (ret != 0) [...]
```

Esempio 2. Thread Multipli

```
#include <stdio.h>
#include <errno.h>
#include <pthread.h>
#define NUM_THREADS 4


void *hello(void *arg) {
      printf («Hello Thread\n»);
}

main() {
  pthread_t tid [NUM_THREADS];
  for (int i = 0; i < NUM_THREADS; i++)
    ret = pthread_create(&tid[i], NULL, hello, NULL);

  for (int i = 0; i < NUM_THREADS; i++)
    ret = pthread_join(tid[i], NULL);
}
```

Esempio 3. Passaggio di Argomenti (lancio di N thread con argomenti necessariamente distinti)

```
#include <errno.h>
#include <pthread.h>

void* foo(void *arg) {
  type_t* obj_ptr = (type_t*) arg; // cast ptr argomenti
  /* <corpo del thread> */
  free(obj_ptr);
  return NULL;
}

int ret;
pthread_t* threads = malloc(N * sizeof(pthread_t));
type_t* objs = malloc(N * sizeof(type_t));
for (i=0; i<N; i++) {
  objs[i] = [...] // imposto argomenti thread i-esimo
  ret = pthread_create(&threads[i], NULL, foo, &objs[i]);
  if (ret != 0) [...]
}
```

Processi

Processi vs Thread

- **Performance**: lanciare/terminare un thread è più veloce rispetto a lanciare/terminare un processo;
- **Memoria**: la creazione di un processo via fork() comporta la copia dell'intera memoria del padre, mentre i thread la condividono;
- **Comunicazione**: i thread di uno stesso processo possono usare la sua memoria per comunicare tra loro, mentre la comunicazione tra processi richiede meccanismi aggiuntivi (Inter-Process Communication).

Misurazione performance

Serve a misurare il **tempo di esecuzione di una porzione di codice**. In fase di compilazione bisogna:

- includere il **modulo performance.c**;

- linkare le **librerie run time** (**-lrt**) e **math** (**-lm**);

Esempio di utilizzo:

```
#include "performance.h"
…
timer t; begin(&t);
// porzione di codice di cui cronometrare l'esecuzione
end(&t); unsigned long int time;
time=get_seconds(&t); time=get_milliseconds(&t);
time=get_microseconds(&t); time=get_nanoseconds(&t);
```

DIPARTI AUTOMA

Tempi di lancio/terminazione

- **Misurazione della reattività** (tempo richiesto) di processi e thread nelle fasi di lancio e terminazione;
- Sorgenti: **reactivity-processes.c**, **reactivity-threads.c**;

Compilazione:

```
gcc -o reactivity-processes reactivity-processes.c
performance.c -lrt -lm
gcc -o reactivity-threads reactivity-threads.c
performance.c -lpthread -lrt -lm
```

In ogni programma vengono eseguiti **N test** per calcolare le r**eattività medie**: N è passato come argomento dalla linea di comando.

Lo **speedup** deve essere calcolato come: $\text{speedup} = \frac{\textit{tempo reactivity process}}{\textit{tempo thread process}}$

Gestione degli errori

errno.h + string.h

Tipicamente, una system call POSIX restituisce -1 per segnalare il fallimento dell'operazione richiesta.

int errno (variabile globale):
- per poterla ispezionare: **#include <errno.h>**;
- in caso di errore, viene dettata con un opportuno codice che indica la causa del fallimento;
- è possibile ottenere una descrizione testuale del codice di errore tramite la funzione **sterror()** da **<string.h>**.

Esempio:

```
#include <errno.h>
#include <stdio.h>
#include <string.h>
...
int ret = my_call(arg1, ..., argN);
if (ret) {
        printf("Error in my_call: %s\n", strerror(errno));
        exit(EXIT_FAILURE);
}
```

perror(const char*str)

```
#include <stdio.h>
#include <errno.h>

void perror(const char *str);
```

- Stampa la stringa **str**, seguita dal messaggio di errore relativo ad errno.
- Se **str** è **NULL** stampa solo il messaggio di errore relativo ad **errno**.
- L'output è stampato sul default error stream.

Macro definiti in common.h

```
// for most common syscalls
#define handle_error(msg) \
do { \
        perror(msg); \
        exit(EXIT_FAILURE); \
} while (0)

// for phtread library only and other functions that do not set errno
#define handle_error_en(ret, msg) \
do { \
        errno = ret; \
        perror(msg); \
        exit(EXIT_FAILURE); \
} while (0)
```

Servono a gestire un errore in una riga sola, invece di riscrivere ogni volta il blocco con **printf** e **exit**.

- **handle_error(msg)**: per le system call più comuni, che in caso di errore dettano **errno.perror(msg)** stampa msg seguito dal messaggio di errore relativo a **errno**, poi **exit(EXIT_FAILURE)** termina il programma.
- **handle_errore_en(ret, msg)**: per la libreria pthread e le altre funzioni che non dettano **errno**, ma restituiscono direttamente il codice di errore. **errno = ret** copia quel codice in **errno**, cosi **perror** può stamparne il messaggio corretto.

Concorrenza in C

Semafori

Le Operazioni sul semaforo sono tre:

1. **Inizializzazione**: assegna un valore iniziale non negativo al semaforo.
2. **Operazione semWait**: decrementa il valore del semaforo; se il valore è negativo il processo/thread viene messo in attesa in una coda, altrimenti va avanti.
3. **Operazione semSignal**: incrementa il valore del semaforo; se il valore non è positivo un processo/thread viene risvegliato dalla coda.

Semafori POSIX

```
#include <semaphore.h>
…
sem_t sem;
…
sem_init(&sem, pshared, value)
…
sem_wait(&sem)
…
sem_post(&sem)
…
sem_destroy(&sem)
```

- **#include <semaphore.h>**: header da includere.
- **sem_t sem**; : dichiarazione di una variabile di tipo **sem_t,** che rappresenta il nostro semaforo.
- **sem_init(&sem, pshared, value)**: inizializzazione del semaforo con valore valore:
  - Se **pshared** vale 0, il semaforo viene condiviso tra i thread del processo.
  - Altrimenti, il semaforo viene condiviso tra processi, a patto che sia in una posizione di memoria condivisa.
  - In caso di successo, viene ritornato 0; in caso di errore, -1.
- **sem_wait(&sem)**: operazione **semWait** sul semaforo **sem**. In caso di successo, viene ritornato 0; in caso di errore, -1.
- **sem_post(&sem)**: operazione **semSignal** sul semaforo **sem**. In caso di successo, viene ritornato 0; in caso di errore, -1.
- **sem_destroy(&sem)**: distrugge il semaforo **sem**. In caso di successo, viene ritornato 0; in caso di errore -1.

Shared Memory

- Offre funzioni per allocare e deallocare una porzione di memoria in modo condiviso.
- La memoria condivisa è mappata su un puntatore.
- Lettura e scrittura vengono effettuate tramite normali operazioni che coinvolgono il puntatore.

Requisiti:

```
#include <sys/mman.h>    /* memory management */
#include <sys/stat.h>    /* mode constants */
#include <sys/types.h>   /* type definition */
#include <fcntl.h>       /* O_* constants */
#include <unistd.h>      /* API of POSIX standard */
```

Link with **-lrt** (Real_time library), for memory management.

shm_open():

```
#include <sys/mman.h>    /* memory management */
#include <sys/stat.h>    /* mode constants */
#include <fcntl.h>       /* O_* constants */

int shm_open(const char *name, int oflag, mode_t mode);
```

- Crea e apre una shared memory, oppure apre una share memory esistente.

- Argomenti:
  - **name**: specifica l'oggetto di memoria condivisa da creare o aprire. Per uso portatile, un oggetto di memoria condivisa deve essere identificato da un nome del tipo "**/name**"; vale a dire una stringa (apparirà il file **/dev/shm/name**).
  - **oflag**: parametri
    i) **O_CREAT**: crea l'oggetto di memoria condivida se non esiste. Il nuovo oggetto di memoria condivisa inizialmente ha una lunghezza pari a zero.
    ii) **O_EXCL**: se è stato specificato anche **O_CREAT** ed esiste già un oggetto di memoria condivisa con il nome specificato \, restituisce un errore.
    iii) **O_RDONLY**: apre l'oggetto per l'accesso in lettura.
    iv) **O_RDWAR**: apre l'oggetto per l'accesso in lettura/scrittura.
    v) Per i nostri scopi:
      - Creazione: **O_CREAT | O_EXCL | O_RDWR**
      - Apertura: **O_RDWR** oppure **O_RDONLY** (secondo necessità)
  - **mode**: permessi utenti. Per i nostri scopi, 0666 oppure 0660.
- Valore di ritorno:
  - In caso di successo, il descrittore della shared memory.
  - In caso di errore, **-1**, **errno** è settato.

ftruncate():

```
#include <unistd.h>      /* API of POSIX standard */
#include <sys/types.h>   /* type definitions */

int ftruncate(int fd, off_t length);
```

- Dimensiona la memoria condivisa a cui fa riferimento **fd** a una dimensione di **length**.
- Se la memoria condivisa in precedenza era più grande di **length**, i dati extra andrebbero persi. Se la memoria in precedenza era più corta, viene estesa e la parte estesa viene letta come byte null (**'\0'**).
- Argomenti:
  - **fd**: descrittore della memoria condivisa ottenuto da **shm_open()**.
  - **length**: dimensione della memoria condivisa.
- Valore di ritorno:
  - In caso di successo, **0**.
  - In caso di errore, **-1**, **errno** è settato.

Curiosità: può essere applicato su un descrittore di file e la dimensione del file cambia di conseguenza.

mmap():

```
#include <sys/mman.h>    /* memory management */

void *mmap(void addr[.length], size_t length, int prot, int flags, int fd, off_t offset);
```

- Mappa la shared memory nella memoria riservata al processo.
- Argomenti:
  - **addr**: permette di suggerire al kernel dove posizionare la memoria condivisa. Se **NULL** o **0** (nostra scelta: 0) il kernel decide autonomamente.
  - **length**: dimensione della memoria condivisa.
  - **fd**: descrittore della memoria condivisa ottenuto da **shm_open()**.
  - **offset**: permette di mappare la memoria condivisa da una posizione diversa da quella iniziale. **offset** deve essere un multiplo della page size. Per noi 0.

  - **prot**: specifica le protezioni sulla modalità di accesso per il processo chiamante. Non deve essere in conflitto con i parametri dettati in **shm_open()**:
    I) **PROT_READ**: permesso di lettura;
    II) **PROT_WRITE**: permesso di scrittura
    III) **PROT_EXEC**: permesso di esecuzione
    IV) **PROT_NONE**: nessun permesso.
    V) Per i nostri scopi **PROT_READ**, **PROT_READ | PROT_WRITE**, **PROT_WRITE**.
  - **flags**:
    i) **MAP_SHARED**: rende le modifiche effettuate nella memoria condivisa visibili agli altri processi.
    ii) Altri flag esistono, ma non sono di nostro interesse.
- Valore di ritorno:
  - In caso di successo, il puntatore all'area di memoria dove risiede la shared memory.
  - In caso di errore **MAP_FAILED**, **errno** è settato.

Curiosità: può essere applicato su un descrittore di file. Il file è mappato in memoria ed è possibile accedere al suo contenuto tramite il puntatore all'area della memoria, invece che tramite le normali operazioni su file. Utile quando il file contiene dati strutturati.

munmap():

```
#include <sys/mman.h>    /* memory management */

int munmap(void *addr, size_t length);
```

- Cancella il mapping tra il processo e la memoria condivisa.
- Successivi tentativi di accesso tramite il puntatore falliranno.
- Argomenti:
  - **addr**: il puntatore alla memoria condivisa ottenuto da **map**.
  - **length**: dimensione della memoria condivisa.
- Valore di ritorno:
  - In caso di successo, **0**.
  - In caso di errore, **-1**, **errno** è settato.

shm_unlink():

```
#include <sys/mman.h>    /* memory management */
#include <sys/stat.h>    /* mode constants */
#include <fcntl.h>       /* O_* constants */

int shm_unlink(const char *name);
```

- Rimuove una memoria condivisa.
- Una volta che tutti i processi hanno fatto l'unmap della memoria, disloca e distrugge il contenuto della regione di memoria associata.
- Successivi tentativi di aprire la memoria falliranno (a meno che non venga usata l'opzione **O_CREAT**).
- Argomenti:
  - **name**: identificatore della memoria condivisa, stesso usato in **shm_open()**.
- Valore di ritorno:
  - In caso di successo, **0**.
  - In caso di errore, **-1**, **errno** è settato.

close():

```
#include <unistd.h>      /* API of POSIX standard */

int close(int fd);
```

- Chiude il descrittore della memoria condivisa.
- Dopo aver effettuato **mmap**, il descrittore può essere chiuso in ogni momento senza influenzare il mapping della memoria.
- Argomenti:
  - **fd**: descrittore della memoria condivisa, ottenuto da **shm_open()**.
- Valore di ritorno:
  - In caso di successo, **0**.
  - In caso di errore, **-1**, **errno** è settato.

Scrivere: write some data in shared memory

```
int main() {
    const int SIZE = 4096;                  /* size of the shared page */
                                    /* name of the shared page */
    const char * name = "/MY_PAGE";
    const char * msg = "Hello World!";
    int shm_fd;
    char * ptr;

    shm_fd = shm_open(name, O_CREAT | O_RDWR, 0666);
    ftruncate(shm_fd, SIZE);
    ptr = (char *) mmap(0, SIZE, PROT_WRITE,
          MAP_SHARED, shm_fd, 0);
    sprintf(ptr, "%s", msg);
    munmap(ptr, SIZE);
    close(shm_fd);
    return 0;
}
```

Il processo scrittore crea la memoria condivisa e ci scrive un messaggio.

- **SIZE = 4096**: dimensione della regione (una pagina); **name = "/MY_PAGE"**: il nome con cui i due processi la identificano; **msg**: il testo da scrivere.
- **shm_open(name, O_CREAT | O_RDWR, 0666)**: crea l'oggetto (se non esiste) e lo apre in lettura/scrittura, con permessi **0666**. Restituisce il descrittore **shm_fd**.
- **ftruncate(shm_fd, SIZE)**: porta la dimensione da 0 a 4096 byte. Senza questo passaggio la memoria sarebbe vuota e non si potrebbe scrivere.
- **mmap(0, SIZE, PROT_WRITE, MAP_SHARED, shm_fd, 0)**: mappa la regione nello spazio del processo. **0** lascia scegliere l'indirizzo al kernel, **PROT_WRITE** permette la scrittura, **MAP_SHARED** rende le modifiche visibili agli altri processi, l'ultimo **0** è l'offset. Restituisce il puntatore **ptr**.
- **sprintf(ptr, "%s", msg)**: scrive il messaggio nella memoria condivisa, come se **ptr** fosse un normale buffer.
- **munmap(ptr, SIZE)** e **close(shm_fd)**: si scollega dalla memoria e chiude il descrittore. Non fa **shm_unlink**, quindi l'oggetto resta nel sistema, con il messaggio dentro, per il lettore.

Leggere: read some data from shared memory

Il processo lettore apre la stessa memoria e stampa il messaggio.

- Stessi **SIZE** e **name**: devono essere gli stessi dello scrittore, altrimenti non trova l'oggetto.
- **shm_open(name, O_RDONLY, 0666)**: apre l'oggetto già esistente in sola lettura (qui non serve **O_CREAT**; il **mode** viene ignorato se l'oggetto non viene creato).

- **mmap(0, SIZE, PROT_READ, MAP_SHARED, shm_fd, 0)**: lo mappa, con permesso di sola lettura (coerente con **O_RDONLY**).
- **printf("%s\n", ptr)**: legge la stringa dal puntatore e la stampa («Hello World!»).
- **munmap**, **close** e infine **shm_unlink(name)**: dopo aver finito, si scollega, chiude il descrittore e rimuove l'oggetto dal sistema. Per questo è il lettore, che arriva per ultimo, a fare l'unlink.

```
int main() {
    const int SIZE = 4096;                  /* size of the shared page */
                                    /* name of the shared page */
    const char * name = "/MY_PAGE";
    int shm_fd;
    char * ptr;

    shm_fd = shm_open(name, O_RDONLY, 0666);
    ptr = (char *) mmap(0, SIZE, PROT_READ,
          MAP_SHARED, shm_fd, 0);
    printf("%s\n", ptr);
    munmap(ptr, SIZE);
    close(shm_fd);
    shm_unlink(name);
    return 0;
}
```

File Descriptor

I file descriptor (FD) sono un'astrazione per accedere a file o altro risorse di input/output come pipe e socket. Ogni processo ha una tabella dei descrittori associata:
- Standard input: 0;
- Standard output: 1;
- Standard error: 2.

Le operazioni di apertura (o creazione) di una risorsa di input/output sono legate al tipo della risorsa stessa:
- **open()** per i file;
- **pipe()** per le pipe;
- **socket()** o **accept()** per le socket.

Lettura

La funzione **read()** è definita in **unistd.h**:

```
ssize_t read(int fd, void *buf, size_t nbyte);
```

- **fd** : descrittore della risorsa;
- **buf**: puntatore al buffer dove scrivere il messaggio letto;
- **nbyte**: numero massimo di byte da leggere.

Ritorna il numero di byte realmente letti, o -1 in caso di errore:
- per i file, ritorna 0 in caso di end-of-file;
- per le socket ritorna 0 in caso di connessione chiusa.

Il buffer deve essere dimensionato per contenere **nbyte** byte. Tale dimensione dipende dall'applicazione:
- formato del file da leggere;
- messaggi da scambiare in un protocollo di comunicazione.

Lettura: interrupt

- L'esecuzione della funzione **read()** ha una certa durata:
  - per i file, richiede il tempo necessario per leggere fino a **nbytes** byte;
  - per le pipe è piuttosto breve perché legge in memoria.
- Nel tempo tra l'invocazione e il termine della **read()**, la chiamata può essere interrotta da un segnale:
  - se l'interruzione avviene prima di riuscire a leggere qualsiasi dato (zero byte letti), la **read()** ritorna -1 ed **errno** viene settato a **EINTR**.
- Può capitare che la read sia interrotta prima che siano ricevuti **nbytes** bytes:
  - la **read()** ritorna il numero di byte letti fino a quel momento (byte letti > 0).
- Una corretta implementazione deve riconoscere queste situazioni ed invocare di nuova la **read()** per ritentare/completare la lettura.

Lettura: esempio

```
while (<not all bytes have been read>) {
    // read from fd up to n bytes and store into buf
    int ret = read(fd, buf, n);

    // no more bytes to read, quit
    if (ret == 0) break;

    if (ret == -1) {
        if (errno == EINTR) continue;   /* interrupted before reading any byte, retry */
        /* error_handler */             // an error occurred...
    }

    /* if interrupted when less than n bytes were read, pay attention to where to
       write on buf on resume! */
    <do something with read bytes>
}
```

Scrittura

La funzione **write()** è definita in **unistd.h**:

```
ssize_t write(int fd, const void *buf, size_t nbyte);
```

- **fd**: descrittore della risorsa;
- **buf**: puntatore al buffer contenente il messaggio da scrivere;
- **nbyte**: numero massimo di byte da scrivere.

Ritorna il numero di byte realmente scritti, o **-1** in caso di errore.

Gestione degli interrupt analoga alla **read()**:
- in caso di interrupt prima di aver scritto il primo byte, viene ritornato **-1** e settato **errno** a **EINTR**;
- in caso di interrupt dopo aver scritto almeno un byte, viene ritornato il numero di byte realmente scritti.

Scrittura: esempio

```
while (<not all bytes have been written>) {
    // write to fd up to n bytes from buf
    int ret = write(fd, buf, n);

    if (ret == -1) {
        // interrupted before writing any byte, retry
        if (errno == EINTR) continue;

        // an error occurred...
        exit(EXIT_FAILURE);
    }

    /* if interrupted when less than n bytes were written, pay attention
       to where you start reading from in the buffer on resume */
    <do something>
}
```

INDICE

Thread in C **1**

Primitive 1

Processi **2**

Processi vs Thread 2

Misurazione performance 2

Tempi di lancio/terminazione 3

Gestione degli errori **3**

errno.h + string.h 3

perror(const char*str) 4

Macro definiti in common.h 4

Concorrenza in C **4**

Semafori 4

Semafori POSIX 5

Shared Memory **5**

Scrivere: write some data in shared memory 8

Leggere: read some data from shared memory 8

File Descriptor **9**

Lettura 9

Lettura: interrupt 10

Lettura: esempio 10

Scrittura 10

Scrittura: esempio 10